

Software Engineering

Prof. Prahlada Rao BB

Dept. of CSE, NIT, Manipur

L1

22-07-2026

Introduction to Software Engineering

CSE Department
National Institute of Technology Manipur- Imphal

1

Software Engineering

Introduction to Software Engineering

Dr. Prahlada Rao B B

ANRF PM Professor, NIT Manipur

L1

22-07-2026

CSE Department
National Institute of Technology Manipur- Imphal

2

Syllabus

CS3501

SOFTWARE ENGINEERING

L-T-P-C 3-0-0-3

Course Outcome

After completing this course, the student will be able to:

- CS3501.1 Understand the principles of software engineering and software processes
- CS3501.2 Apply requirements engineering and modeling techniques using UML and design concepts
- CS3501.3 Analyze quality measures, metrics, and estimation techniques for software projects
- CS3501.4 Evaluate software testing methods and risk management strategies
- CS3501.5 Develop project schedules and apply maintenance strategies for software systems

3

3

Syllabus

Syllabus:

- Software and Software Engineering: The Software Process;
- Process Models: Modeling: Requirements Engineering,
- Requirements Modeling, UML, Design Concepts, Etc.;
- Quality Management; Product Metrics: Process and Project Metrics;
- Software Estimation Techniques;
- Software Testing Strategies: Project Scheduling; Risk Management; Maintenance.

Texts :

1. Rajib Mall, Fundamentals of Software Engineering, 5th Edition Prentice Hall India
2. Pressman, R.S., Software Engineering: A Practitioner's Approach, McGraw Hill, seventh edition, 2010.

References:

1. Sommerville, Ian, Software Engineering, Addison-Wesley, fifth edition, 2000.
2. Jalote, P., An Integrated Approach to Software Engineering, Narosa Publishing House, second edition, 2003.
3. Bennett S., McRobb S. & Farmer R., Object Oriented Systems Analysis and Design using UML, TataMcGraw-Hill, second edition, 2004.

4

Organization of this Lecture

- What is Software Engineering?
- Programs vs. Software Products
- Evolution of Software Engineering
- Notable Changes in Software Development Practices
- Introduction to Life Cycle Models
- Summary

5

What is Software Engineering?

- Engineering approach to Develop Software.
 - Building Construction Analogy.
- Systematic collection of past experience:
 - Techniques.
 - Methodologies.
 - Guidelines.



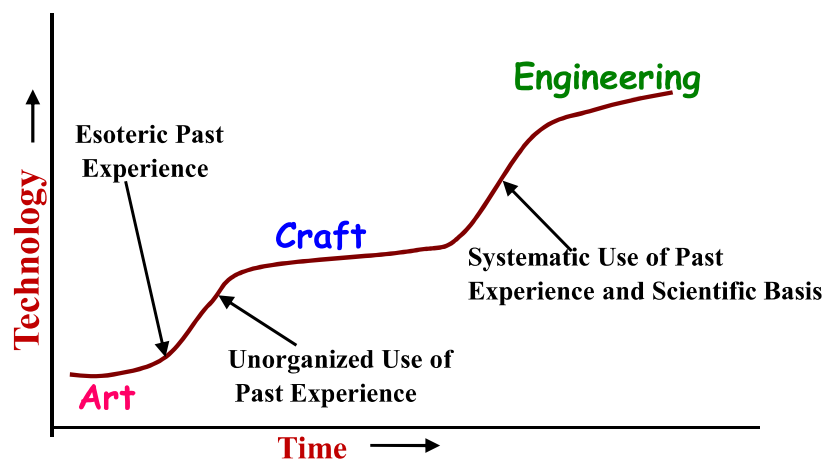
6

Engineering Practice ?

- Heavy use of past experience:
 - Past experience is systematically arranged.
- Theoretical basis and Quantitative Techniques provided.
- Many are just thumb rules.
- Trade-off between alternatives.
- Pragmatic approach to cost-effectiveness.

7

Technology Development Pattern



8

Evolution of an Art into an Engineering Discipline

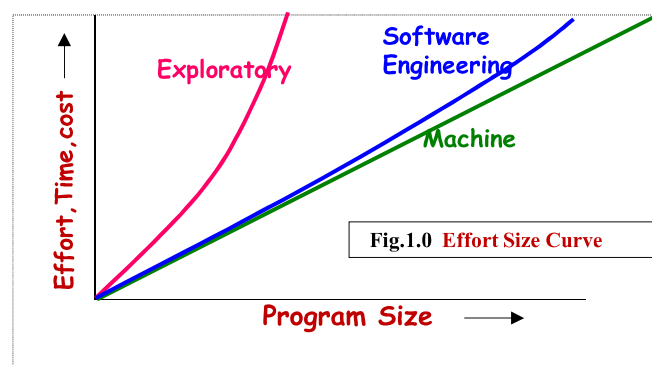
- The early programmers used an exploratory (also called **build and fix**) style.
 - In the build and fix (exploratory) style, normally a '**dirty**' program is quickly developed.
 - The different imperfections that are subsequently noticed are **fixed**.

9

What is Wrong with Exploratory Style?

- Can successfully be used for very small programs only.

Fig.1.0 Effort Size Curve.



10

What is Wrong with the Exploratory Style?

- Exponential growth of effort, cost & time with problem size:
 - Exploratory style usually results in unmaintainable code.
 - It becomes very difficult to use the exploratory style in a team development environment.
- Why does the effort required to develop a product grow exponentially with product size?
- Why the approach completely breaks down when the product size becomes large?

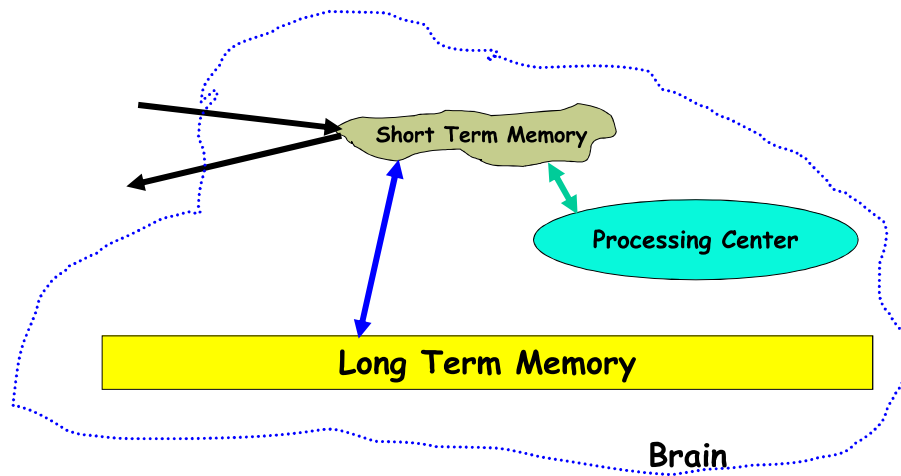
11

An Interpretation Based on Human Cognition Mechanism

- Human memory can be thought to be made up of two distinct parts [Miller 56]:
 - Short term memory and
 - Long term memory.
- If you are asked question: "If it is 10AM now, how many hours are remaining today?"
 1. First, 10AM would be stored in the short-term memory.
 2. Next, a day is 24 hours long would be fetched from the long-term memory into short-term memory.
 3. Finally, mental manipulation unit would compute the difference (24-10).

12

Human Cognition Mechanism



13

Short Term Memory

- An item stored in the short-term memory can get lost:
 - Either due to decay with time or
 - Displacement by newer information.
- This restricts the time for which an item is stored in short term memory to few tens of seconds.
- However, an item can be retained longer in the short term memory by recycling.

14

What is an Item?

- An item is any set of related information.
 - A character such as 'a' or a digit such as '5' can be items.
 - A word, a sentence, a story, or even a picture can each be a single item.
- Each item normally occupies one place in memory.
- When you are able to relate several different items together (chunking):
 - The information that should normally occupy several places can be stored using only one place in the memory.

15

Chunking

- If you are given the binary number 110010101001.
 - It may prove very hard to understand and remember.
 - But octal form of 6251 (i.e (110)(010)(101)(001)) would be easier.
 - You have managed to create chunks of three items each.

16

Evidence of Short-Term Memory

- Short-term memory is evident:
 - In many of our day-to-day experiences.
- Suppose, you look up a number from the telephone directory and start dialling it.
 - If number is busy, you can dial the number again after a few seconds, effortlessly without having to look up the directory.
- But, after several days:
 - You may not remember the number at all, and would need to see the directory again.

17

The Magical Number 7

- If a person deals with seven or less number items:
 - Can easily be accommodated in the short-term memory.
 - So, he can easily understand it.
- As number of new information increases beyond seven:
 - It becomes exceedingly difficult to understand it.

18

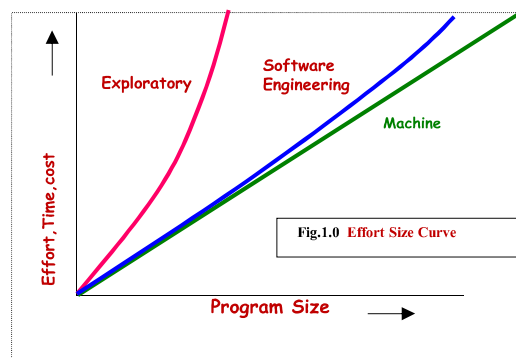
Implication in Program Development

- A small program having just a few variables:
 - Is within the easy to grasp of an individual.
- As number of independent variables in program increases:
 - It quickly exceeds the grasping power of an individual:
 - Requires large effort to master the problem.

19

Implication in Program Development

- Instead of a human, if a machine is writing (gen) a program:
 - The slope of the curve would be linear.
- The effort-size curve become almost linear when SE principles are deployed. But why ?
 - Software Engg. principles extensively use techniques, to overcome the human cognitive limitations.



20

Principles used by Software Engineering to Overcome Human Cognitive Limitations

- Mainly two important principles are deployed:
 - Abstraction
 - Decomposition

21

Abstraction

- Simplify a problem by omitting unnecessary details.
 - Focus attention on only one aspect of the problem and ignore irrelevant details.
- Suppose you are asked to develop an overall understanding of some country.
 - No one in his right mind would meet all the citizens of the country, visit every house, and examine every tree of the country, etc.
 - You would possibly refer to various types of maps for that country.
- A map, is an abstract representation of a country.

22

Decomposition

- **Decompose** a problem into many **small independent parts**.
 - The **small parts** are then taken up one by one and solved separately.
 - **The idea is** that each **small part** would be **easy to grasp** and can be **easily solved**.
 - The **full problem** is solved when all the **parts** are solved.

23

Decomposition

- **A popular way to demonstrate the decomposition principle:**
 - Try to break a bunch of sticks tied together versus breaking them individually.
- **Example: Use of decomposition principle:**
 - You **understand a book better** when the **contents** are organized into independent chapters
 - Compared to when everything is mixed up.

24

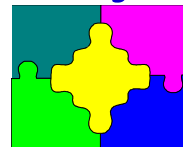
Why Study Software Engineering? (1)

- To acquire skills to develop large programs:
 - Exponential growth in complexity and difficulty level with size.
 - The adhoc approach breaks down when size of software increases.

25

Why Study Software Engineering? (2)

- Ability to solve Complex Programming Problems:
 - How to break large projects into smaller and manageable parts?
 - How to use abstraction?
- Also learn techniques of:
 - Specification, design, user interface development, testing, project management, etc.
- To acquire skills to be a better programmer:
 - Higher Productivity
 - Better Quality Programs



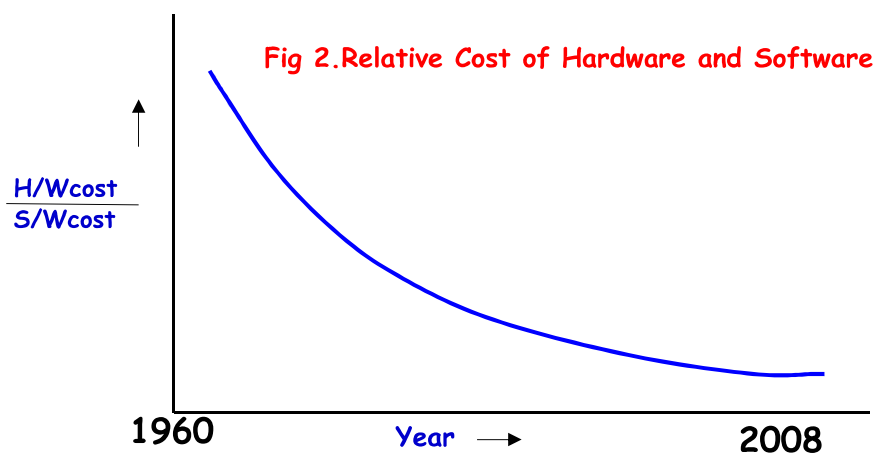
26

Software Crisis ?

- Sometimes Software Products:
 - Fail to meet user requirements.
 - Frequently crash.
 - Expensive.
 - Difficult to alter, debug, and enhance.
 - Often delivered late.
 - Use resources non-optimally.

27

Software Crisis (cont.)



28

Factors Contributing to Software Crisis

- Larger problems
- Lack of adequate training in Software Engineering
- Increasing skill shortage
- Low productivity improvements.

29

Programs versus Software Products

Programs	Software Products
• Usually small in size	• Large
• Author himself is sole user	• Large number of users
• Single developer	• Team of developers
• Lacks proper user interface	• Well-designed interface
• Lacks proper documentation	• Well documented & user-manual prepared
• Ad hoc development.	• Systematic development

22-07-2026: Upto Here →

30

Types of Software Projects

- Software Products
- Outsourced Projects
- Indian companies, focused on Outsourced Projects

31

Computer Systems Engineering ?

- Computer Systems Engineering:
 - Encompasses Software Engineering.
- Many products require development of software as well as specific hardware to run it:
 - A coffee vending machine,
 - A mobile communication product, etc.

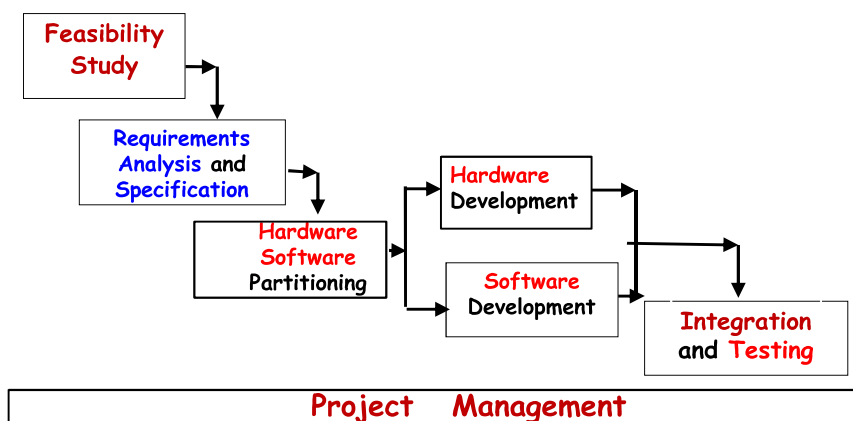
32

Computer Systems Engineering

- **High-level Decision Problem:**
 - Deciding **which tasks** to be solved by **Software**.
 - Which ones **by Hardware**.
- **Often, Hardware and Software are developed together:**
 - **Hardware Simulator** is used during **Software Development**.
- **Integration of Hardware and Software.**
- **Final System Testing**

33

Computer Systems Engineering



34

Emergence of Software Engineering

- **Early Computer Programming (1950s):**
 - Programs were being written in assembly language.
 - Programs were limited to a few hundreds of lines of assembly code.
- Every programmer developed his own style of writing programs:
 - According to his intuition (Exploratory Programming).

35

High-Level Language Programming (Early 60s)

- HLLs sa FORTRAN, ALGOL, and COBOL were introduced:
 - This reduced Software Development Efforts greatly.
- Software development style was still exploratory.
 - Program sizes were limited to a few thousands of lines of source code.

36

Control Flow-Based Design (late 60s)

- Size and Complexity of programs increased further:
 - Exploratory programming style proved to be insufficient.
- Programmers found:
 - Very difficult to write cost-effective and correct programs.

37

Control Flow-Based Design (late 60s)

- Programmers found:
 - Programs written by others very difficult to understand and maintain.
 - To cope up with this problem, experienced programmers advised:
 - Pay attention to the design of Programs Control Structure.
- A Program's Control Structure indicates:
 - The sequence in which program's instructions are executed.
 - To help design programs having good control structure:
 - Flow Charting technique was developed.

38

Control Flow-Based Design (late 60s)

- Using Flow Charting Technique:
 - One can represent and design a program's control structure.
 - Usually one understands a program:
 - By mentally simulating the program's execution sequence.
- A program having a messy flow chart representation:
 - Difficult to understand and debug.

39

Control Flow-Based Design (Late 60s)

- It was found:
 - GO TO statements makes control structure of a program messy.
 - GO TO statements alter the flow of control arbitrarily.
 - Need to restrict use of GO TO statements recognized.

40

Control Flow-Based Design (Late 60s)

- Many programmers had extensively used assembly languages.
 - JUMP instructions are used for program branching in Assembly Languages.
 - Programmers considered use of GO TO statements inevitable.
- At that time, Dijkstra published his article:
 - "Goto Statement Considered Harmful" Comm. of ACM, 1969.
- Many programmers were unhappy to read his article.
- They published several counter articles:
 - Highlighting the advantages and inevitability of GO TO statements.

41

Control Flow-Based Design (Late 60s)

It was conclusively proved:

- Only three programming constructs are sufficient to express any programming logic:
 - Sequence (Ex. A=0;b=5;)
 - Selection (Ex. If(c=true) k=5 else m=5;)
 - Iteration (Ex. While(k>0) k=j-k;)

Everyone accepted:

- It is possible to solve any programming problem without using GO TO statements.
- This formed the basis of Structured Programming Methodology.

42

Structured Programming

- A program is called **Structured**: When it uses only the following types of constructs:
 - **Sequence**,
 - **Selection**,
 - **Iteration**

43

Structured Programs

- **Unstructured Control Flows** are avoided.
 - **Consist of a neat set of modules.**
 - **Use single-entry, single-exit program constructs.**
-
- **Violations to this feature are permitted: Due to practical considerations such as:**
 - **Premature loop exit** to support **Exception Handling**.

44

Structured programs

- **Structured programs are:**
 - Easier to read and understand.
 - Easier to maintain.
 - Require less effort and time for development.
- **Research experience shows:**
 - Programmers commit less nu.of errors:
 - While using structured if-then-else and do-while statements.
 - Compared to test-and-branch constructs.

27-07-2026: Upto Here → 45

Data Structure-Oriented Design (Early 70s)

- **Discovered, 'Important to pay more attention to':**
 - The design of data structures of a program, than to
 - The design of control structure.
- **Techniques which emphasize, designing data structure:**
 - Derive program structure from it, and are called:
 - Data Structure-oriented Design Techniques.

46

Data Structure Oriented Design (Early 70s)

- **Ex. Data Structure oriented design Technique:**
 - Jackson's Structured Programming (JSP) methodology, Developed by Michael Jackson in 1970s.
- **JSP technique:**
 - Program Code Structure should correspond to, The Data Structure.

47

Data Structure Oriented Design

(Early 70s)

- **In JSP methodology:**
 - A program's data structures are first designed using notations for: Sequence, Selection and Iteration.
 - Then data structure design is used: To derive the program structure.
- Several data structure-oriented methodologies also exist:
 - Ex. Warnier-Orr Methodology

48

Data Flow-Oriented Design (Late 70s)

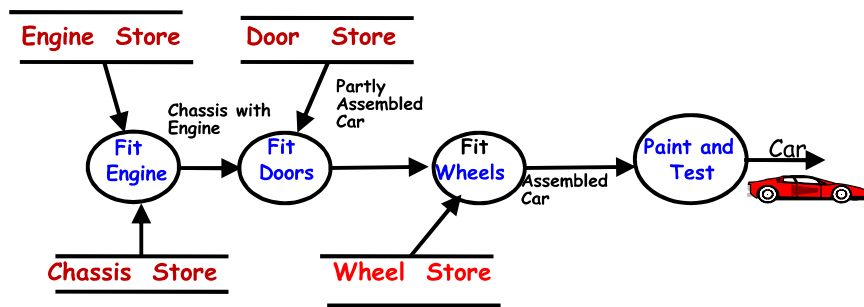
- Data flow-oriented techniques advocate:
 - Data items input to a system must 1st be identified,
 - Processing required on the data items to produce the required outputs should be determined.
- Data flow technique identifies:
 - Different processing stations in a system.
 - The items (data) that flow between processing stations.

49

Data Flow-Oriented Design (Late 70s)

- Data flow technique is a generic technique:
 - Can be used to model the working of any system.
 - not just software systems.
- Major advantage of data flow technique is its simplicity.

Data Flow Model of a Car Assembly Unit:



50

Object-Oriented Design (80s)

Object-oriented technique:

- An intuitively appealing design approach:
- **Natural objects** (s.a **employees**, **pay-roll-register**, etc.) occurring **in a problem** are first identified.
- **Relationships** among objects (s.a **composition**, **reference**, and **inheritance**) are determined.
- **Each object essentially acts as**
 - **A data hiding** (or **data abstraction**) **entity**.

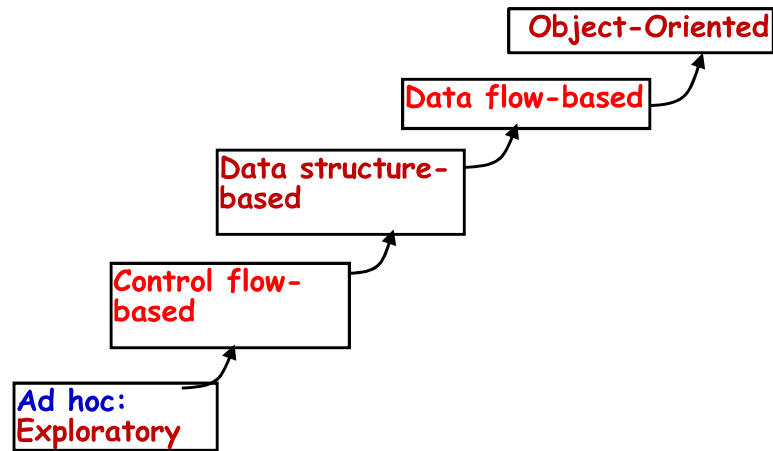
51

Object-Oriented Design (80s)

- **Object-Oriented Techniques** have gained wide acceptance:
 - **Simplicity**
 - **Reuse possibilities**
 - **Lower development time and cost**
 - **More robust code**
 - **Easy maintenance**

52

Evolution of Design Techniques:



53

Evolution of Other Software Engineering Techniques

- The improvements to Software Design methodologies
 - Are indeed very conspicuous.
- In additions to software design techniques:
 - Several other techniques evolved.

54

Evolution of Software Engineering Techniques

- Life cycle models
- Specification techniques
- Project management techniques
- Testing techniques
- Debugging techniques
- Quality assurance techniques
- Software measurement techniques
- CASE tools, etc.

55

Modern Software Development Practices

- Use of Life Cycle Models.
- Software is developed through several well-defined stages:
 - Requirements analysis and specification,
 - Design,
 - Coding,
 - Testing, etc.

56

Differences bet'n exploratory style and modern Software Development Practices

- **Emphasis has shifted:**
 - from **error correction** to **error prevention**.
- **Modern practices emphasize:**
 - **Detection of errors, as close** to their **point of introduction** as possible.

57

Differences bet'n exploratory style and modern software development practices (CONT.)

- **In exploratory style:**
 - **Errors are detected** only during **Testing**.
- **Now,**
 - **Focus** is on **detecting as many errors** as possible in each phase of development.
- **In exploratory style:**
 - **Coding** is **synonymous with** program development.
- **Now,**
 - **Coding** is considered only a **small part of** program development effort.

58

Modern Software Development Practices

- A lot of effort and attention is now paid to:
 - Requirements specification.
- There is a distinct design phase:
 - Standard design techniques are being used.
- During all stages of development process:
 - Periodic reviews are being carried out
- Software testing has become systematic:
 - Standard testing techniques are available.

59

Differences bet'n exploratory style and modern Software Development Practices (CONT.)

- There is better visibility of design and code:
 - Visibility means production of good quality, consistent and standard documents.
- Increased visibility makes Software Project Management easier.
- In the past, very little attention was given to producing good quality and consistent documents.

60

Differences bet'n exploratory style and modern Software Development Practices (CONT.)

- Because of good documentation:
 - Fault diagnosis and maintenance are smoother now.
- Several metrics are being used to:
 - Help in software project management, quality assurance, etc.

61

Differences bet'n exploratory style and modern Software Development Practices (CONT.)

- Projects are being thoroughly planned:
 - Estimation.
 - Scheduling.
 - Monitoring mechanisms.
- Use of CASE tools.

62

Software Life Cycle ?

- **Software Life Cycle (or Software Process):**
Series of identifiable stages that a software product undergoes during its life time:
 1. Feasibility study
 2. Requirements Analysis and Specification,
 3. Design
 4. Coding
 5. Testing
 6. Maintenance

63

Life Cycle Model

Software Life Cycle Model (or Process Model):

- A descriptive diagrammatic model of Software Life Cycle:
 - Identifies all activities required for product development.
 - Establishes a precedence ordering among the different activities.
 - Divides life cycle into phases.
- Several activities may be carried out in each life cycle phase.
- For Ex: The Design Stage might consist of:
 - Structured Analysis activity followed by
 - Structured Design activity.

64

Why Model Life Cycle ?

- **A written description:**
 - **Forms a common understanding of activities among the software developers.**
 - **Helps in identifying inconsistencies, redundancies and omissions in development process.**
 - **Helps in Tailoring a Process Model for specific projects.**
- **Processes are tailored for special projects.**
 - **A documented Process Model**
 - **Helps to identify where the tailoring is to occur.**

65

Life Cycle Model

- **Development team must identify a life cycle model:**
 - **and then adhere to it.**
- **Advantage of adhering to a life cycle model:**
 - **Helps development of software in a systematic and disciplined manner.**

66

Life Cycle Model (CONT.)

- When a program is developed by single programmer:
 - He has the freedom to decide his exact steps.
- When a software product is developed by a team:
 - There must be a precise understanding among team members as to when to do what ?
 - Otherwise it would lead to chaos and project failure.
- A Software Project will never succeed if:
 - One engineer starts writing code,
 - Another concentrates on writing test document first,
 - Yet another engineer first defines the file structure
 - Another defines the I/O for his portion first.

67

Life Cycle Model (CONT.)

- A Life Cycle Model:
 - Defines entry and exit criteria for every phase.
 - A phase is considered to be complete:
 - Only when all its exit criteria are satisfied.
- Phase Exit criteria for Software Requirements Specification (SRS) is:
 - Software Requirements Specification (SRS) document is complete, reviewed and approved by the customer.
- Phase can start:
 - Only if its Phase Entry criteria have been satisfied.

68

Life Cycle Model (CONT.)

- It becomes easier for software project managers:
 - To monitor the progress of the project.
- When a life cycle model is adhered to, the project manager can at any time fairly accurately tell:
 - at which stage (e.g design,code,test) the project is.
 - Otherwise, it becomes very difficult to track the progress of the project
 - Project manager would have to depend on guesses of the team members.
- This usually leads to a problem:
 - known as the 99% complete syndrome.

69

Life Cycle Model (CONT.)

- Many life cycle models have been proposed.
- We confine attention to important & commonly used models.
 - Classical waterfall model
 - Iterative waterfall,
 - Evolutionary,
 - Prototyping, and
 - Spiral model

70

Summary

Software Engineering is:

- Systematic collection of decades of programming experience
- Together with the innovations made by researchers.
- A fundamental necessity while developing any large software product.
- Adoption of a life cycle model.

Adherence to a Software Life Cycle model:

- Helps to do various development activities in a systematic and disciplined manner.
- Also makes it easier to manage a software development effort.